Chapter 3

Ontology

In this chapter we discuss the topic of ontologies, which is a cornerstone of the work developed throughout this thesis.

We start, in section 3.1, by giving a historical perspective on ontology from a philosophical point of view. First, we present the different definitions of the term ontology in philosophy, and, subsequently, discuss the importance of ontological investigations for science, in general, and for conceptual modeling, in particular.

The past decades have observed an increasing interest in ontologies in a wide range of computer-related applications. Section 3.2 discusses four areas that, historically, have been prominently responsible for creating the demand for the use of ontologies in computer science, namely, information systems, domain engineering, artificial intelligence and the semantic web. For each of these areas we briefly discuss the expected benefits and provide examples of ontologies and ontology-based applications that have been developed along the years. In addition, we discuss the characteristics of ontology modeling languages that are typically used in some of these areas.

Although ontology has a clear definition in philosophy, there is a substantial terminological confusion in the different areas of computer science regarding what the term is supposed to denote. Section 3.3 is the most important section of this chapter and aims at a terminological clarification and at establishing a formal characterization of the way the term is used in the remaining of this work. Furthermore, the section elaborates on the relation between the definition provided and the notions of conceptualization and language as discussed in chapter 2.

In section 3.4, we present some final considerations.

3.1 Ontology in Philosophy

The Webster dictionary (Merriam-Webster, 2004) defines the word ontology as:

(D1). a branch of metaphysics concerned with the nature and relations of being;
(D2). a particular theory about the nature of being or the kinds of existents;
(D3). a theory concerning the kinds of entities and specifically the kinds of abstract entities that are to be admitted to a language system.

The term ontology was coined in the 17th century in parallel by the philosophers Rudolf Göckel in his *Lexicon philosophicum* and by Jacob Lorhard in his *Ogdoas Scholastica* (figure 3.1). The term *Ontologia*, however, was popularized in philosophical circles only in 18th century by the publication in 1730 of the *Philosophia prima sive Ontologia* by Christian Wolff (figure 3.2).

Etymologically, *ont-* comes from the present participle of the Greek verb *einai* (to be) and, thus, the latin word ***Ontologia*** (*ont-* + *logia*) can be translated as *the study of existence*.

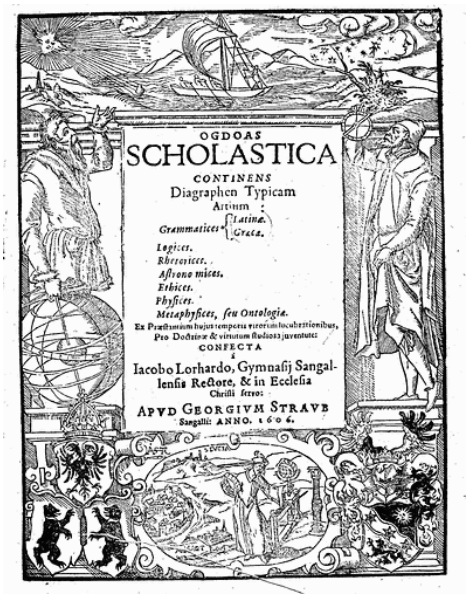

OGDOAS
SCHOLASTICA
CONTINENS
Diagraphen Typicam
Artium :
Grammatices { Latinæ. / Græcæ.
Logices.
Rhetorices.
Aſtrono micæs.
Ethices.
Phyſices.
Metaphyſices, ſeu Ontologiæ.
Ex Præſtantium hujus temporis virorum lucubrationibus,
Pro Doctrinæ & virtutum ſtudiosa juventute:
CONFECTA
A
Iacobo Lorhardo, Gymnaſij Sangal-
lenſis Rectore, & in Eccleſia
Chriſti ſervo:
APVD GEORGIVM STRAVB
Sangallis ANNO. 1606.

Figure 3-1 Cover of Jacob Lorhard's book *Ogdoas Scholastica* from 1606


PHILOSOPHIA
PRIMA,
SIVE
ONTOLOGIA,
METHODO SCIENTIFICA
PERTRACTATA,
QUA
OMNIS COGNITIONIS
HUMANÆ PRINCIPIA
CONTINENTUR.
AUTORE
CHRISTIANO WOLFIO,
EDITIO NOVA
PRIORI EMENDATIOR.

Figure 3-2 Cover of Christian Wolff's book *Philosophia prima sive Ontologia* from 1730

In the sense (D1) of the aforementioned Webster definition, ontology is the most fundamental branch of metaphysics. Aristotle was the first western philosopher to study metaphysics systematically and to lie out a rigorous account of ontology. He described (in his *Metaphysics* and *Categories*) ontology as *the science of being qua being* (Corazzon, 2004). According to this view, the business of ontology "…is to study the most general features of reality and real objects" (Peirce, 1935), i.e., the study of the generic traits of every mode of being. As opposed to the several specific scientific disciplines (e.g., physics, chemistry, biology), which deal only with entities that fall within their respective domain, ontology deals with transcategorical relations, including those relations holding between entities belonging to distinct domains of science, and also by entities recognized by ***common sense***.

Ontology aims to develop theories about, for example, persistence and change, identity, classification and instantiation, causality, among others. Ontological questions include questions such as: what kinds of entities exist? What differentiates objects from events and how are they related? What are the properties of a thing and how are they related to the thing itself? What is the essence of an object? Does essence precede existence? Are things bundles of properties? Is an object equal to the sum of its parts? Are there Natural Kinds? Is change possible without a changing thing? These are general but factual questions, only comprehensive rather than specific. They are also fundamental to science regardless if we are to talk about properties of atoms, human organs or insurance claims, if we are to develop theories of physical, mental or social events, or if we are to theorize

on the parthood relations between an individual and a society, a heart and a human body and the first half and an entire football game.

There are many ontological principles that are utilized in scientific research, for instance, in the selection of concepts and hypothesis, in the axiomatic reconstruction of scientific theories, in the design of techniques, and in the evaluation of scientific results. Examples of scientific questions that are actually metaphysical ones include (Bunge, 1977, p.19):

- Is there an ultimate matter? This question triggered Heisenberg's 1956 theory of elementary particles;
- Is society anything beyond and above individuals that compose it or are there special societal laws in addition to laws governing individual behavior? This is a central dispute in the methodology and philosophy of social sciences;
- Are biological species embodiments of Platonic archetypes, or just concrete populations? Or perhaps a single individual scattered in space and time? This question is asked everyday by taxonomists (Ereshefsky, 2002).

Examples of metaphysical hypothesis underlying scientific research include (Bunge, 1977, p.16):

- There is a world external to the cognitive subject;
- The world is composed of things that are grouped into systems or aggregates;
- Every system except the universe, interacts with other systems in certain respects and is isolated from other systems in other respects;
- Nothing comes out of nothing and no thing reduces to nothingness;
- Everything changes;
- There are laws and everything abides by laws (otherwise, the experimental scientific method would not be possible);
- There are levels of organization: physical, chemical, social, psychological, biological, etc. The so-called higher levels emerge from other levels in the course of processes; but, once formed they enjoy a certain stability with laws of their own. Otherwise we would have to know everything about physics and chemistry before knowing about organisms and societies.

Moreover, recent scientific theories that make explicit use of metaphysical concepts abound. To cite one example, in (Penrose, 2000), the author uses a Platonic theory of Universals to explain the mind's ability to execute non-computable processes.

Finally, in the axiomatization of scientific theories, some of the following concepts are to occur in an explicit fashion: part, composition, system, state of affairs, relations, boundary, causality, state, event, change, property, law, possibility, process, space and time. However, the specific axioms of these theories will usually not tell us anything about these fundamental and generic concepts. Science just borrows them, leaving them in an intuitive and pre-systematic state. In other words, these generic concepts are common to a number of sciences, so that no single scientific discipline takes the trouble to regiment them (Bunge, 1977). The same holds from concepts in computer science and, in particular, in conceptual modeling. Concepts such as part and whole, instantiation and classification, attribution and relationships, causality, interaction, among others, are represented in the modeling primitives of several conceptual modeling language or, at minimum, are used in discourse of the computer science literature. Nonetheless, there is a lack of theoretical support in the area for precisely defining the meaning of these concepts. To quote Ron Weber, in his *Ontological Foundations of Information Systems* (Weber, 1997): *"A discipline that calls itself the information systems discipline ought to know what the term 'information systems' means…[, and for that to take place,] its members need to be able to explain precisely what they mean by the term 'system'."*

In summary, every science presupposes some metaphysics. However, metaphysics and science can be distinguished by the scope of their problems. Whereas the scientist deals with rather specific questions of fact, the ontologist is concerned with all the factual domains.

In the beginning of the 20th century the German philosopher Edmund Husserl coined the term ***Formal Ontology*** as an analogy to Formal Logic. Whilst Formal Logic deals with formal logical structures (e.g., truth, validity, consistency) independently of their veracity, Formal Ontology deals with formal ontological structures (e.g., theory of parts, theory of wholes, types and instantiation, identity, dependence, unity), i.e., with formal aspects of objects irrespective of their particular nature. The unfolding of Formal Ontology as a philosophical discipline aims at developing a system of general categories and their ties, which can be used in the development of scientific theories and domain-specific common sense theories of reality. In other words, ontology in the first sense of Webster's definition aforementioned contributes to the development of ontologies in the second sense.

The first ontology developed in sense (D2) is the set of theories of Substance and Accidents developed by Aristotle in his *Methaphysics* and *Categories*[7]. Since then, ontological theories have been developed by

[7] We refer to (Aristotle, 1984) for a modern edition of Aristotle's complete works.

innumerous philosophers such as G.W. Leibniz (Leibniz, 1981), C.S. Peirce (Peirce, 1935), Alfred North Whitehead (Whitehead, 1929), Bertrand Russel (Russel, 1940), Willen Van Orman Quine (Quine, 1953, 1969), Nelson Goodman (Goodman, 1951), Peter Strawson (Strawson, 1959), Edmund Husserl (Husserl, 1970), and Saul Kripke, (Kripke, 1982) to name just a few whose theories appear latter in this thesis. Moreover, philosophical ontology is currently an active area in philosophy, and formal ontologies have been built by contemporary philosophers such as Mario Bunge (Bunge, 1977), Eli Hirsch (Hirsch, 1982), Anil Gupta (Gupta, 1980), Jacques van Leeuwen (van Leeuwen, 1991), Rodderick Chisholm (Chisholm, 1996), David Armstrong (Armstrong, 1989, 97), E.J. Lowe (Lowe, 2001), Peter Simons (Simons, 1987), Peter Gärdenfors (Gärdenfors, 2000), David Wiggins (Wiggins, 2001), Kevin Mulligan (Mulligan & Simons & Smith, 1984), Barry Smith (Smith, 1998), Achille Varzi and Roberto Casati (Casati & Varzi, 1994). Finally, more recently a number of ontological systems in the sense (D2) have been constructed in projects related to computer science (Masolo et.al, 2003a; Heller & Herre, 2004), and under the auspices of a new discipline called *Applied Ontology* (Masolo et.al, 2003b). It is with this last kind of ontologies that this thesis is mainly concerned, i.e., with formal ontological theories that can be developed and *applied* in the solution to problems in the fields of computer and information sciences and, in particular, of conceptual modeling.

3.2 Ontology in Computer and Information Sciences

Since the word ontology has been mentioned in a computer related discipline for the first time (Mealy, 1967), ontologies have been applied in a multitude of areas in computer science. The first noticeable growth of interest in the subject in mid 1990's was motivated by the need to create principled representations of domain knowledge in the knowledge sharing and reuse community in AI, which motivated the creation of forums such as the conference series FOIS (Formal Ontology and Information Systems)[8]. Nonetheless, an explosion of works related to the subject only happened in the past two years. Just to illustrate this point, the paper submission rate from the first International Semantic Web Working (SWWS) Symposium in 2001 (Cruz, 2001) to the third edition of the International Semantic Web Conference (ICSW) (Fensel & Sycara & Mylopoulos, 2003) has increased by almost 400%.

[8] http://www.fois.org/

According to (Smith & Welty, 2001), historically there are three areas mainly responsible for creating a demand for the application of ontologies in computer science, namely, (i) database and information systems; (ii) software engineering (in particular, domain engineering); (iii) artificial intelligence. In the sequel, we discuss the use of ontologies in these areas. Additionally, we also include a discussion of the topic in the context of the Semantic Web, due to the important role played by this area in the current popularization of the term.

3.2.1 Ontology in Information Systems

According to (Smith, 2004), the term *"ontology"* in the computer and information science literature appeared for the first time in 1967, in a work on the foundations of data modeling by S. H. Mealy, in a passage where he distinguishes three distinct realms in the field of data processing, namely: (i) *"the real world itself"*; (ii) *"ideas about it existing in the minds of men"*; (iii) "*symbols on paper or some other storage medium".* Mealy concludes the passage arguing about the existence of things in the world regardless of their (possibly) multiple representations and claiming that *"This is an issue of ontology, or the question of what exists"* (Mealy 1967. p. 525.). In the end of this passage, Mealy includes a reference to Quine's essay *"On What There Is"* (Quine, 1953).

The fields of data and information modeling have been a fruitful ground for the applications of ontological theories, either implicitly or explicitly. In the 1970s, the so-called three schema architecture has been proposed in the database field (Jardine, 1976). The architecture suggests the distinction between *implementation schemas* (describing physical ways of storing data and procedures), *presentation schemas* (concerned with external interfaces to the user), and *conceptual schemas* (focusing of the description of the characteristics of the elements in the universe of discourse). In order to address the problem of how to create these conceptual schemas, different set of modeling concepts have been proposed.

First, the so-called *logical models* were proposed in early 1970's. Examples include the relational and network models. These models offered a set of abstract modeling primitives that were independent of physical modeling concepts but which, unfortunately, were deemed flat (i.e., lacking structure[9]) and unintuitive as how they should be used for modeling purposes (Mylopoulos, 1998).

Soon after logical models were proposed, different meta-conceptualizations were proposed which offered more expressive facilities

[9] See discussion on section 3.4.2 of this chapter.

for modeling applications and structuring information bases. The *semantic model* proposed by Jean-Raymond Abrial in 1974 (Abrial, 1974) and Peter Chen's *entity-relationship model* (Chen, 1976) are included in the category of languages that are now known as conceptual modeling (or semantic data modeling) languages.

The creation of both logical and conceptual models by the database and information modeling community was solely motivated by the search for better concepts that could be used for creating representations of a certain portion of reality. Both the Semantic model and the ER model were committed to a world view and based on the *ontological assumption* that the structural aspects of the world could be articulated by using the concepts of *entity* and *relationship*. However, none of these efforts took ontology seriously, in the sense that the choices of categories that are part of the conceptualization underlying these languages were not based on Ontology in the philosophical sense.

As pointed by (Smith & Welty, 2001), the *ad hoc* and inconsistent modeling that marked the early days of conceptual modeling led to many of the practical database integration problems that we face today. In order to tackle some of these problems and to provide a sound basis for the selection of modeling concepts that should underpin information system grammars, several researchers started to found their work on philosophical ontologies. Examples include the use of philosophical ontologies for analysis and evaluation of: (i) *information systems grammars* (Milton & Kamierczak, 2004; Shanks & Tansley & Weber, 2003; Evermann and Wand, 2001a,b; Gemino, 1999; Green and Rosemann, 2000; Opdahl and Henderson-Sellers, 2001; Opdahl & Henderson-Sellers & Barbier, 1999; Parsons and Wand, 1991; Wand and Weber, 1989, 1993; Wand & Storey & Weber, 1999); (ii) *reference models* (Fettke and Loos, 2003); (iii) *data quality* (Wand and Wang, 1996); (iv) *off-the-shelf system (COTS)* (Soffer et. al, 2001).

Along the years, there has been an increasing interest in the use of philosophical ontology in the conceptual modeling and information systems community as a foundation for their discipline. Research results following this idea are strongly related to the objectives of this work and will be thoroughly discussed in the development of this dissertation.

3.2.2 Ontology in Domain Engineering

Independently of these developments in the information systems community, yet another sub-field of computer science, namely software engineering, began to recognize the importance of what came to be known as *domain engineering* (Arango & Williams & Iscoe, 1991). This was mainly motivated by the need to reduce the disproportional costs in software

maintenance and the need to reinforce software reuse in a higher level of abstraction than merely programming code (Arango, 1994).

In general, a domain engineering process is composed of the following subactivities: *domain analysis* and *domain design*, the latter being further decomposed in *infrastructure specification*, *infrastructure implementation*. Intuitively, domain engineering can be considered analogous to software engineering (software application engineering), however, operating in a meta-level (see table 3.1), i.e., instead of uncovering requirements, designing and implementing a specific application, the target is on a family of applications in a given domain (Arango & Prieto-Diaz, 1994).

Table 3-1 A comparison between Domain Engineering and Application Engineering activities (based Arango & Prieto-Diaz, 1994)

Application Engineering	Domain Engineering
Requirements Analysis	Domain Analysis
Application Design	Infrastructure Specification
Application Implementation	Infrastructure Implementation

The term *Domain Analysis* appears for the first time in (Neighbors, 1981) with the following definition: *"Domain Analysis is an attempt to identify objects, operations and relations that domain experts perceive as important in a given domain"*.

The product of a domain analysis phase is a *domain model*. A domain model defines objects, events and relations that capture similarities and regularities in a given domain of discourse. The resulting model is an architecture comprised of conceptual components that are common to a family of applications (*reuse*). It can be used to identify, explain and predict facts in a given domain, which can hardly be observed directly (*problem-solving*). Moreover, it serves the purposes of a unified reference model to be used when ambiguities arise in discussions about the domain (*communication*) and a source of knowledge that can be used in a learning process about that domain. In summary, the specification produced by the domain modelling activity is *"a shared representation of entities that domain experts deem relevant in a universe of discourse, which can be used to promote problem-solving, communication, learning and reuse in a higher level of abstraction"* (Arango, 1994).

Actually, more than a mere analogy, domain and application engineering are complementary disciplines and can be interrelated in a process that contemplates both development *to reuse* and development *with reuse.* Figure 3.3 depicts schematically how these disciplines are integrated in the so-called *two-level life cycle* (Falbo et. al, 2002a). Some of these relations are briefly elaborated below.

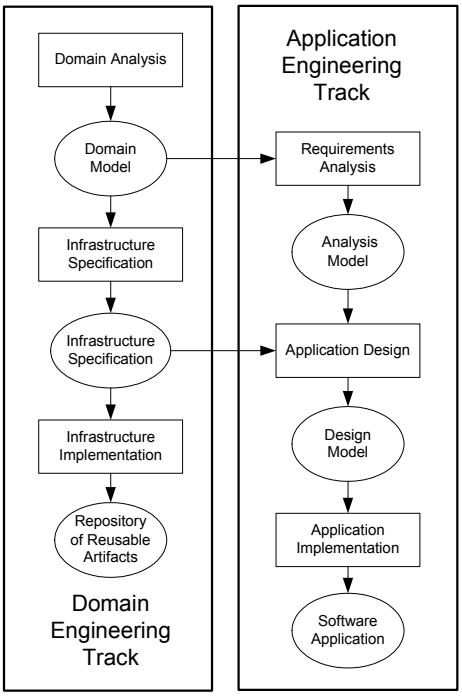


Figure 3-3 Domain Engineering and *Application Engineering* (based on Falbo et al., 2002a)

The result of a domain engineering process is a reusable infrastructure or *framework* (development for reuse). A framework can be (re)utilized in instances of software engineering processes for the construction of a several specific applications (development with reuse) whose requirements have been defined during the requirements analysis phase of each specific application.

In addition to being the basis for the development of framework, a domain model can also be used in the application requirements analysis phase to improve communication and understanding of the domain and to help in the requirements elicitation process (Falbo & Menezes & Rocha, 1998).

For a framework to be representative and, therefore, useful for potential applications in a domain, it must embed a correct conceptualization of the entities perceived as important by domain experts. The same must be true for a domain model to be useful as a shared reference for communication and problem-solving in application requirements analysis. Analogous to the problem faced by conceptual modelers in the database community, the challenge in *domain modeling* is again *finding the best concepts that can used to create representations of phenomena*

in a universe of discourse that are both as reusable as possible and still truthful to reality.

This field, too, was severely debilitated by a lack of concrete and consistent formal bases for making modeling decisions (Smith & Welty, 2001). Despite the strong correspondence between domain models and what is named *domain ontologies* in AI (see section 3.2.3), only very recently ontologies started being used as a foundation for domain engineering. For instance, (Falbo et. al, 2002a) presents an ontology-based approach for software reuse and discusses how ontologies can support several tasks of a reuse-based software process. In (Falbo & Guizzardi & Duarte, 2002), an ontological approach for domain engineering (named ODE) is advocated and a systematic approach for ontological engineering is proposed. (Guizzardi & Falbo & Pereira Filho, 2001a), (Guizzardi & Falbo & Pereira Filho, 2001b) and (Guizzardi & Falbo & Pereira Filho, 2002) propose a modeling language for building ontology-based domain models and a systematic approach for deriving object-oriented frameworks from them. The framework derivation methodology proposed comprises a spectrum of techniques, namely, mapping directives, design patterns and formal translation rules. In particular, (Guizzardi & Falbo & Pereira Filho, 2001b) introduces a design pattern for guaranteeing the preservation of some ontological properties of part-whole relations (irreflexivity, asymmetry, transitivity and shareability) in object-oriented implementations.

(Falbo & Guizzardi & Duarte, 2002) and (Guizzardi & Falbo & Pereira Filho, 2002) exemplify the approaches proposed by constructing ontologies and deriving the corresponding object-oriented frameworks for the domains of software process and software quality, respectively. Fragments of these ontologies are depicted in figure 3.4 and 3.5, in that order. The representation language LINGO used in the picture is a set-theoretical language proposed in (Falbo & Menezes & Rocha, 1998) and (Guizzardi & Falbo & Pereira Filho, 2001a). The complete boxes represent concepts, boxes with open sides represent (material) relations, arrows represent subsumption relations (the arrow head pointing to the most general concept) and (hollow) circles represent shareable part-whole relations (the circle being connected to the whole. The cardinality constraints adjacent to a concept constrain the opposed navigational end of the relation. For example, in figure 3.4, a *Measurable Quality Characteristic can be measured by one-to-many Metrics*. Moreover, whenever omitted, cardinality constraints should be interpreted as zero-to-many. Finally, grey boxes represent concepts imported from other ontologies. For example, in the ontology of figure 3.5, the concept *Software Process* is imported from the Software Process ontology of figure 3.4. This picture also shows an axiom in this ontology which states that: *the resources allocated to a compound software activity*

A are those (and only those) which are allocated to the (sub)activities that are proper-parts of A.

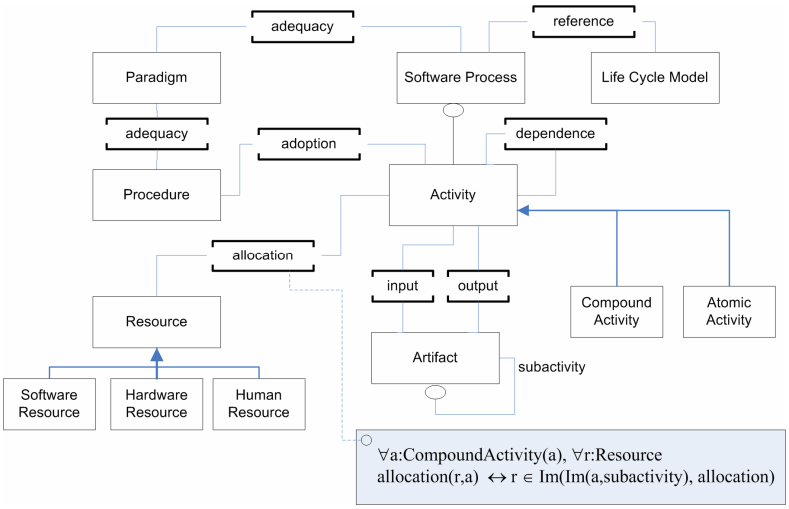


Figure 3-4 Excerpts of a Software Process Ontology developed using ODE and LINGO

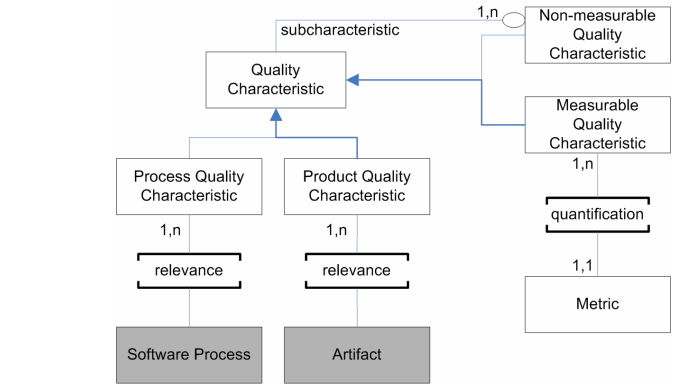


Figure 3-5 Excerpt of a Software Quality Ontology developed using ODE and LINGO

The ontology of figure 3.4 is used to generate the framework shown in figure 3.6. One can observe that, by using the transformation rules proposed by the methodology, the axioms in the domain ontology are systematically and explicitly mapped onto methods in the target framework. In the case exemplified, an invocation of the method *usage* in an *Activity* object returns the solution set of the corresponding axiom in the ontology, i.e., the set of resources used in by that specific *Activity*. This solution increases the reusability of the produced framework by representing explicitly the methods that address the ontology *competence questions*

(Gruninger & Fox, 1994a), as opposed to have domain knowledge hidden inside the code.

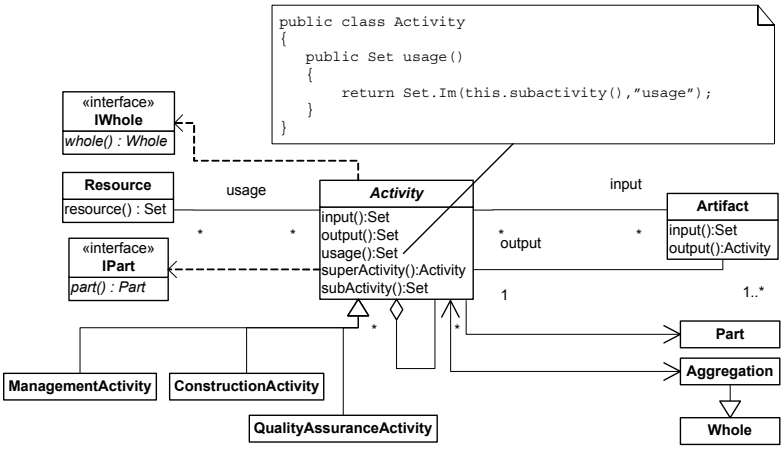


Figure 3-6 Fragment of a *Software Process Framework* derived from a *Software Process Ontology*

This framework can be reused and extended in a software engineering *development with reuse* process to address the needs of specific applications. For example, in (Falbo et. al, 2002b), the frameworks derived from the ontologies in figure 3.4 and 3.5 are used in the development of tools that are integrated in a software engineering environment. In particular, the *process framework* is used in the development of *process definition* (figure 3.7) and *project tracking* tools and the *quality framework* in the development of a *quality control application* (figure 3.8).

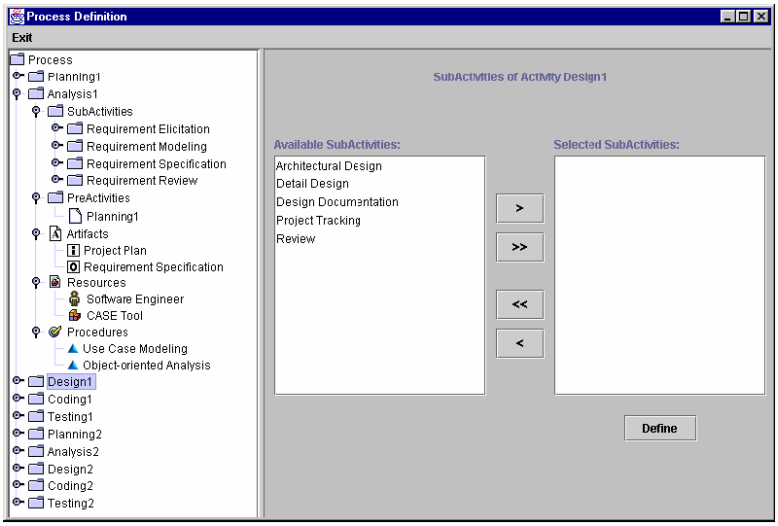


Figure 3-7 Process Definition Tool developed by (re)using a *Software Process Framework*

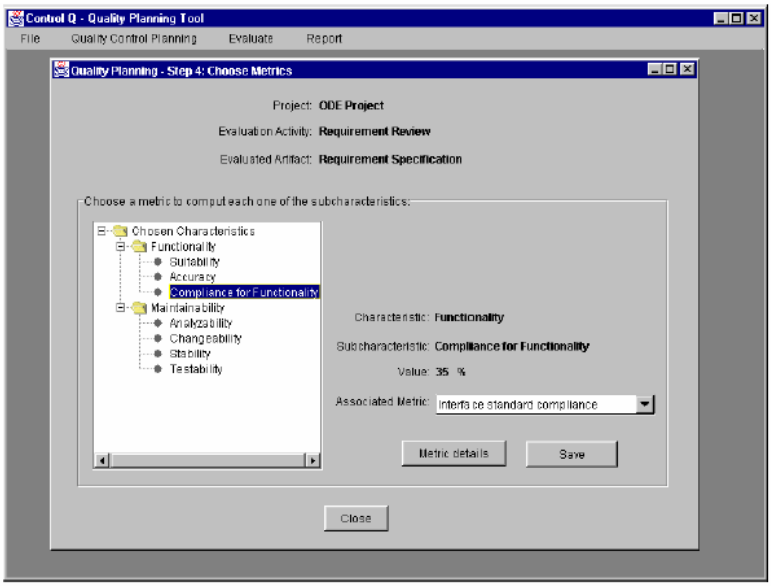


Figure 3-8 Quality Control Tool (Control Q) developed by (re)using a Software Quality Framework

A number of other domain ontologies and frameworks have been developed using ODE and LINGO in domains such as *resource allocation* (Guizzardi & Falbo & Pereira Filho, 2001a), *Software Risk* (Falbo et. al, 2004), *Knowledge Management* (Natali & Falbo, 2002), *Organizational Modeling* (Cota & Menezes & Falbo, 2004), *Steel Factoring* (Mian et. al, 2002), among others. Moreover, the domain engineering methodology proposed has been used as a foundation for the construction of an ontology editor (Mian & Falbo, 2002) and a domain-oriented software engineering environment (Mian & Falbo, 2003). An extension of this approach for enterprise engineering has been proposed as part of the AGILA project (Caplinkas, 2003).

LINGO was designed with the specific objective of achieving a positive trade-off between expression power of the language and the ability to facilitate bridging the gap between the conceptual and implementation levels (a preoccupation that also seem to be present in Peter Chen's original proposal for ER diagrams). The language succeeds in offering abstractions that conform to the object-oriented paradigm, which, hence, enable a systematic translation approach. Moreover, it contains some important ontological distinctions, for example, the distinction between sortal and non-sortal concepts (see chapter 4 of this work). Nonetheless, its purely extensional semantics and ontological incompleteness (in the technical sense proposed in chapter 2) make it inappropriate as a general conceptual modeling language.

An important point that should be emphasized is the difference in the senses of the word ontology used by the information systems and domain engineering communities. In information systems, the term ontology has been used in ways that conform to its definitions in philosophy (in both senses D1 and D2). As a system of categories, an ontology is independent of language: Aristotle's ontology is the same whether it is represented in English, Greek or First-Order Logic. In contrast, in most of other areas of computer science (including domain engineering and *artificial intelligence*), the term ontology is, in general, used as a concrete engineering artifact designed for a specific purpose. In section 3.4, we provide a precise account for this latter use of the term and elaborate on its relation to conceptualization and language as discussed in chapter 2.

Finally, as a concrete artifact, an ontology should be constructed in a systematic process analogous to those of traditional software engineering. An ontological engineering process model typically comprises activities such as: *Purpose Identification and Requirements Specification, Ontology Capture, Formalization, Reuse and Integration, Evaluation and Documentation* (Falbo & Guizzardi & Duarte, 2002). For a more elaborated discussion on ontological engineering methodologies one should refer to (Gruninger & Fox, 1994b; Falbo & Menezes & Rocha, 1998; Fernández-López et. al, 1999; Gomez-Perez & Corcho & Fernandez-Lopez, 2002; Devedžić, 2002).

3.2.3 Ontology and Artificial Intelligence

The work of (Clancy, 1993) was of great importance for laying the groundwork for the establishment of ontology in artificial intelligence. In the tradition of AI, up to that moment, knowledge in artificial systems used to be defined in a strictly *functional* way aiming to incorporate in a knowledge base the steps that domain experts typically use in the solution of a given problem. Clancy proposed a shift in such a perspective, arguing that *"the primary concern of knowledge engineering is modeling systems in the world, not replicating how people think"* (ibid.). Clancy is a proponent of the *modeling view* of knowledge acquisition, according to which a knowledge base is not a repository of knowledge extracted from an expert's mind (as in the so-called *transfer view*), but it refers to an objective reality that is much more related to the classical notion of truth intended as a correspondence to the real-world[10]. More exactly, the modeling activity must establish a

[10] In the correspondence theory of the truth advocated by philosophers such as (Russel, 1918) and (Wittgenstein, 1922), the truth of a proposition is determined by its correspondence to the existence of an entity or fact in reality, the so-called *Truthmaker* of a proposition (Mulligan & Simons & Smith, 1984). Here, by objective reality, we mean a task-

correspondence between a knowledge base and two separate subsystems: the behavior of the intelligent system (i.e., the problem-solving expertise) and its environment (the problem domain) (Guarino, 1995).

Guarino strongly defends the view that the modeling of domain knowledge should be pursued in a way that is as independent as possible of the problem-solving task. The argument is based on two important points:

1. If the represented knowledge is not considered as *part* of the objective reality of the domain, the very basic assumptions of the modeling view are contradicted: if a domain theory does not describe (partially) *"an inherent structure in the domain"*, what is it supposed to represent? Arguably, the agent's mind, which was exactly what the modeling view aimed to avoid (Guarino, 1995, p.2). This reasoning can also be applied when directed to domain models and schemas used in domain engineering and information systems, respectively. How can we make systems with different conceptual models but overlapping semantics work together, if not by referring to the common world to which they all relate?

2. Knowledge acquisition (domain analysis, requirements engineering, etc.) is a notoriously expensive process and, hence, reuse of domain knowledge and domain representations should be maximized across different applications. With such a perspective, knowledge specifications can acquire a value *per se*, and as much as we approximate to the truth as classically conceived the potential for reuse should increase. As put in (Smith, 1995), truth in classical sense is a sort of infinite reusability.

For these reasons, it becomes clear how the study of ontology (in sense D1) can be of great benefit to the knowledge-construction process of intelligent systems, which is a position that finds strong support in (Guarino, 1995, 1997, 1998; Smith, 2004). Nonetheless, as in the case of information systems and domain engineering, the initial approaches to tackle domain modeling in AI suffered from a relative narrow perspective, concentrating on the immediate needs of the AI practice, and refusing to take into account the philosophical achievements coming from the study of common-sense reality (Guarino, 1995).

Based on these considerations, in (Guarino, 1998), the author proposes a classification of ontology kinds based on their level of dependence on a particular task or point of view:

---

independent conceptualization that is *truthful to reality* in the sense discussed in depth in section 3.4.1.

– *Top-level ontologies* describe very general concepts like space, time, matter, object, event, action, etc., which are independent of a particular problem or domain;

– *Domain ontologies* and *task ontologies* describe, respectively, the vocabulary related to a generic domain (like medicine, or automobiles) or a generic task or activity (like diagnosing or selling), by specializing the terms introduced in a top-level ontology;

– *Application ontologies* describe concepts that depend both on a particular domain and task, and often combine specializations of *both* the corresponding domain and task ontologies. These concepts often correspond to *roles* played by domain entities while performing a certain task, like *replaceable unit* or *spare component*.

In this definition, an *application ontology* is not considered as a synonym to a *knowledge base*. An ontology can be considered as a *particular* knowledge base, describing facts assumed, by a community of users, to hold *necessarily*, in virtue of the agreed-upon meaning of the vocabulary used. A generic knowledge base, instead, may also describe facts and assertions related to particular state of affairs or particular epistemic state. Consequently, within a generic knowledge base two components can be distinguished: the ontology (containing situation-independent information) and the "core" knowledge base (containing situation-dependent information) (Guarino & Giaretta, 1995). The idea of application ontologies is analogous to that of *specializations of knowledge frameworks* (Falbo et. al, 2002a,b; Falbo & Guizzardi & Duarte, 2002; Falbo & Menezes & Rocha, 1999). Specializations of knowledge frameworks are created to incorporate task and application knowledge that are dependent of a particular purpose and functionality that should be performed by a specific system or narrower class of systems.

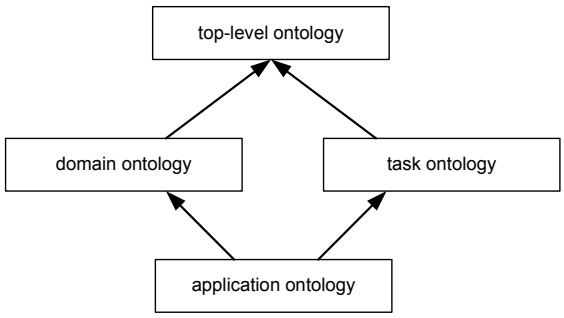


Figure 3-9 A classification of different types of ontology. Arrows represent specialization relations (from Guarino, 1998)

The instances of all these different types of ontologies depicted in figure 3.9 are concrete engineering artifacts. Since the first time the word ontology was used in AI by Hayes (Hayes, 1978) and since the development of his naïve physics ontology of liguids (Hayes, 1985), a large amount of ontologies (mostly domain ontologies) have been developed. In the sequel, we briefly discuss some examples of domain ontologies that have been constructed over the years:

Engineering and Technical Applications

The *YMIR* ontology (Alberts, 1994) is a domain independent, sharable ontology for the formal representation of engineering design knowledge, based on systems theory. *EngMath* (Gruber & Olsen, 1994) is an ontology for mathematical modeling in engineering. It includes representations for scalar, vector, and tensor quantities, physical dimensions, units of measure, functions of quantities, and dimension quantities. *PHYSSYS* ontology (Borst, 1997) is an ontology for modeling, simulating and designing physical systems. It consists of three engineering ontologies that formalize the three viewpoints on physical devices, namely, system layout, physical process behavior and descriptive mathematical relations. *DORPA* (Varejão et. al, 2000) is an ontology for the design of reprographic machines developed in collaboration with the XEROX Palo Alto Research Center.

Enterprise Modeling and Manufacturing

The *TOVE* (*Toronto Virtual Enterprise*) ontology (Gruninger & Atefi & Fox, 2000), formalizes knowledge about production/communication processes, activities, causality, resources, quality and cost in business enterprises. Another example includes the *Enterprise Ontology* developed in a project supported by the UK's department of Trade and Industry and led by the AI Applications Institute at the University of Edinburgh (see Uschold et. al, 1998). It defines a collection of terms (e.g., activities and processes, organization, strategy, marketing) that are considered relevant for business enterprises. Once represented, these enterprise ontologies have been used as a basis for the development of methods and tools for enterprise modeling. The *Process Specification Language* (*PSL*) (Schlenoff et al., 2000; Ciocoiu & Gruninger, 2000) defines a neutral representation for manufacturing processes, which can be used to integrate modeling languages and tools that are used throughout the life cycle of a product, from early design of manufacturing process, through process planning, validation, production scheduling and control.

Chemistry, Biology and Ceramic Materials

CHEMICALS (Fernández-López et. al, 1999) is an ontology that contains knowledge within the domain of chemical elements and crystalline structures. The *Gene Ontology* (GO)[11] project is a collaborative effort to address the need for consistent descriptions of gene products among several of the world's largest repositories for plant, animal and microbial genomes. It is composed of three structured, controlled vocabularies that describe gene products in terms of their associated biological processes, cellular components and molecular functions in a species-independent manner. The *Fishery Core Ontology* (Gangemi et. al, 2002) was developed to support semantic interoperability among existing fishery information systems. Other examples include the *PLINIUS* (van der Vet & Mars, 1994) ontology of ceramic materials and the *ontology of pure substances* (van der Vet & Mars, 1995).

Medicine

The *GALEN* project aims at developing a terminological server for medical concepts (Rector at. al, 1995). It is based on a semantically sound model of clinical terminology: the *GALEN Coding reference* (*CORE*) *model*. This model comprises elementary clinical concepts (e.g., fracture, bone, and humerus), and relationships (e.g., *fractures can occur in bones*) that control how these may be combined. Moreover, it includes complex concepts (such as *fracture of the left humerus*) composed from simpler ones. The *ON9* is a large-scale ontology library for medical terminology developed in the context of the *ONIONS* (*ONtological Integration of Naïve Sources*) project (Gangemi & Pisanelli & Steve, 1999). One of terminological sources that are formalized and integrated in ON9 is the *UMLS* Metathesaurus developed by the American National Library of Medicine (Pisanelli & Gangemi & Steve, 1998). Additionally, (Pisanelli et. al, 2003) describes an ontology representing the concepts involved in evidence-based medicine and meta-analysis

Law

The *Core Legal Ontology* (*CLO*) (Gangemi et. al, 2003) is used to organize juridical concepts and to represent the assessment of legal regulatory compliance across different legal systems or between norms and cases. In (Sagri & Tiscornia & Gangemi, 2004) CLO is used as a foundation for the development of *JurWordNet* (a semantic lexicon to support legal information

[11] http://www.geneontology.org/

searching) (Sagri, 2003), and for the representation of click-on licenses in the field of Intellectual Property Right (IPR).

3.2.4 Ontology and the Semantic Web

The World Wide Web has been made possible due to the availability of a set of well established standards that guarantee interoperability at various levels, like, for instance, the transport and application level (with the TCP[12] and HTTP[13] standards, respectively), and the presentation level (with the HTML standard[14]). After Tim Berners-Lee's seminal paper (Berners-Lee, 1989-1990), the web started to be collectively created as a hypermedia network of information nodes. Along these years, it suffered a transformation from a medium for *information exchange* to a medium for *service deployment*. Nonetheless, its current structure is still directed for human processing and interpretation. Regardless if the user is booking a flight, searching for news or analysing catalogues of Volvo parts made by different manufactures, the interpretation of the content of each web node relies totally on the user's ability to give real-world semantics to symbols written in a certain language.

The next developmental step for the web has been characterized by the so-called *Semantic Web* vision (Berners-Lee, Hendler, Lassila, 2001), which stipulates a change in the web from being *machine-readable* (but only human-understandable) to *machine-understandable*. By machine-understandability, Berners-Lee and colleagues mean the following: web resources (information nodes and computational services) are annotated by meta-data written in a formal knowledge representation language, i.e., in a language with precisely defined formal semantics and with associated highly optimized inference procedures and engines. As a consequence, techniques developed by the knowledge representation community over the years could be exploited in the development of intelligent services, such as intelligent search engines (Mayfield & Finin, 2003), information brokering and filtering (Klien et. al, 2004; Vögele & Hübner & Schuster, 2003), web service annotation and automatic service composition/orchestration (Majithia & Walker & Gray, 2004; Paolucci & Sycara & Kawamura, 2003), knowledge management architectures (Davies & Fensel & van Harmelen, 2003; Guizzardi & Aroyo & Wagner, 2003), among many others.

In this scenario, ontologies are expected to play a fundamental role by interweaving human understanding of symbols with machine processability. The idea is that, firstly, ontologies representing shared conceptualizations of

[12] Transmission Control Protocol (http://www.faqs.org/rfcs/rfc793.html)
[13] HyperText Transfer Protocol (http://www.w3.org/Protocols/)
[14] http://www.w3.org/MarkUp/

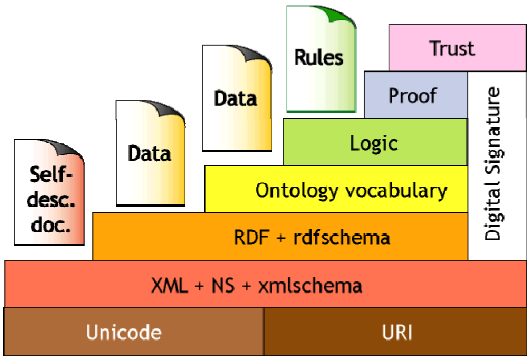
reality are constructed and specified in a *machine-understandable* language. Then, as formal specifications, they can be used as semantic domains for the definition of formal and real-world semantics for syntactic symbols present in web resources.

In the sequel, we discussed the most relevant semantic web technologies from the point of view of domain representation.

3.2.4.1 An architecture for the Semantic Web

One of the main architectural premises of the Semantic web is the stack of languages, often drawn in a figure firstly presented by Berners-Lee in his XML 2000 keynote address[15] and replicated in figure 3.10. In the sequel we briefly discuss some layers in this stack leading up to the ontology representation languages. For a more detailed discussion one should refer to (Koivunen & Miller, 2001).

Figure 3-10 The Semantic Web Layered Architecture



In the bottom language layer of the stack we have the *eXtensible Markup Language* (*XML*)[16]. As opposed to HTML documents, which have a predefined syntactic structure, XML was designed as a markup-language for arbitrary document structures. By using *XML Schema Definitions* (*XSD*'s), one can define a grammar (an abstract syntax definition) for a class of XML documents, defining vocabulary and syntactic rules that are specific to a class of applications. For this reason, it can be used as serialization syntax for other markup languages. For example, the *Synchronized Multimedia Integration Language* (*SMIL*)[17] is syntactically just a particular XSD.

For a processing application, any symbol in a HTML document that is not a member of the predefined markup primitives is seen as a meaningless

[15] Available at http://www.w3.org/2000/talks/1206-xml2k-tbl/.
[16] http://www.w3.org/XML/
[17] http://www.w3.org/AudioVideo/

string of characters. In contrast, XML provides some structuring and a standard way for defining the abstract syntax for a class of documents that can be shared by other applications. Nonetheless, a XSD only specifies syntactic rules, and any intended semantics for the syntactic elements is totally left outside the realm of the XML specification. Hence, for a processing application, the tags used to structure a XML document are as devoid of meaning as a variable label in a Java program, regardless of the natural language term that we give to it.

On top of the XML layer, the W3C[18] defines the *Resource Description Framework* (*RDF*)[19], which standardizes the definition and use of meta-data descriptions about web resources. A web resource, from a RDF point of view, is anything that can be given a *Uniform Resource Identifier* (*URI*)[20], such as a web page, a computer device, a multimedia file, etc. Examples of applications of metadata descriptions are:

- Website map (the metadata structure can describe the content of the web pages as well as their interrelationships);
- Descriptions of resources privacy policies;
- Description of advisory rating for multimedia content;
- Description of device capabilities;
- Description of user's preferences and characteristics;
- Digital Signatures;
- Content classification systems (in particular, its ability to describe taxonomic relationships can be exploited, for example, for books, films, articles, etc.).

RDF data is made up of statements, where each statement expresses the values of the properties of a resource. The basic RDF data model consists of an object-attribute-value triple commonly written as *Attribute(Object,Value)*. For example, in figure 3.11, we present a RDF description in the notation of labelled directed graphs. In this figure, the property title (predicate) relates the http://www.example.org/ resource (subject) with a literal (object).

[18] W3C stands for *World Wide Web Consortium* which is the entity responsible for the standardization of WWW-related technologies (http://www.w3.org/).
[19] http://www.w3.org/RDF/
[20] http://www.gbiv.com/protocols/uri/rfc/rfc3986.html/

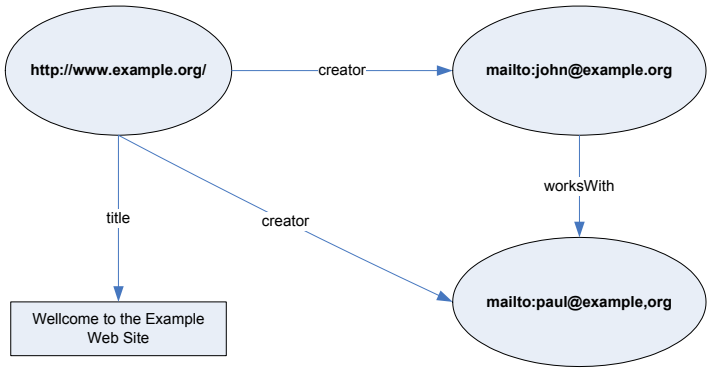


Figure 3-11 Example of a simple RDF description

In principle, RDF could also be used to represent any ordinary data model. However, it comprises only a minimalist model containing primitives such as *description*, *resource*, *property*, and does not provide any means for the definition of the terms used in resource annotation, such as *creator*, *workswith* and *title* in figure 3.11. For this reason, a schema language named *RDF Schema* (*RDFS*)[21] has been proposed. In contrast with XML, the RDF/RDFS model is able to define the semantics of its specifications (in case, for instance, a semantics such as the one proposed in (Pan & Horrocks, 2003)[22] is adopted).

In the semantic web community, RDFS is regarded as a simple ontology representation language, since it incorporates the simplest parts of frame-based languages such as *OKBC* (Grosso, 1999) (i.e., *classes, properties, domain and range restrictions, instance-of, subclass-of* and *subproperty-of* relationships). Nonetheless, there are many useful types of formulas that cannot be expressed in the language. A few examples are:

- In a RDFS specification, one cannot state that two subconcepts of a common concept form a partition, i.e., that they are disjoint and complete;
- although restrictions can be posed on the type of resources that form the domain and range of a property, cardinality constraints cannot be represented in a RDFS specification.

In order to overcome the deficiencies of RDFS as an ontology representation language, two different languages were proposed, offering a

[21] http://www.w3.org/TR/rdf-schema/

[22] An interesting aspect of the non-standard model theoretical semantics defined in this paper is its ability to solve some ambiguity problems with the RDF model as well as its compatibility with layering of a description logic based ontology modelling language (e.g. OWL) on top of it.

richer set of modelling primitives: *DAML-ONT* (McGuiness et al., 2002a) and *OIL* (Fensel et. al, 2001). Later, these languages converged in a single proposal named *DAML+OIL* (McGuiness et al., 2002b), which was further refined to become the W3C recommendation *OWL* (*Ontology Web Language*) (Bechhofer et. al, 2004). These (logical layer) ontology modelling languages have been carefully designed for the best possible trade-off between expressiveness and computational efficiency, *the latter property having precedence*. The language inherits from a specific type of family of *description logics* (*SHIQ*) (Baader & Horrocks & Sattler, 2003; Horrocks & Patel-Schneider & van Harmelen, 2003) its formal semantics and reasoning support, which ensure logical completeness, correctness and efficiency.

Another design requirement for the language was related to the need for maintaining compatibility with other languages in the stack of figure 3.10. Consequently, OWL has been provided with both XML and RDF serializations. On one hand, an OWL specification is a valid XML document whose syntax is property defined in XSD. On the other hand, OWL reuses many of the RDFS primitives (e.g., *class, domain, range, property*), which makes its specifications partially available to RDFS-only software.

Finally, the language was also designed aiming at being intuitive to the human user. For this reason its modelling primitives are based on the popular paradigm of frame-based and object-based languages. A related (but nonetheless diverse) point, which has not been considered by the designers of the language, is the pragmatic effectiveness of the language's concrete syntax. Although mainly targeted for machine-processing (as opposed to human problem-solving and communication), OWL specifications are, in general, created and manipulated by humans. In order to tackle this problem some proposals towards a UML syntax for OWL have been pursued (see, for example, Baclawski et. al, 2001).

3.2.4.2 Context-Aware Services: An application Scenario

An application area in which Semantic web ontologies have been employed successfully is the subfield of ubiquitous computing named context-aware computing (Ríos, et. al, 2003; Chen & Finin & Joshi, 2003; Strang & Linnhoff-Popien & Korbinian, 2003). Context-aware computing is a new computing paradigm that has brought the possibility of exploring the dynamic context of the user in order to provide more adaptable, complex and personalized services. A context is a situation involving the user or its environment, which can be considered of interest for an application. Examples of contexts include:

– Physical contexts (such as location and time, etc.);

- Environmental contexts (weather, altitude, velocity, humidity, light, etc.);
- Informational contexts (stock quotes, sports scores, etc.);
- Personal contexts (health, mood, schedule, activity, etc.);
- Social contexts (group activity, social relationships, vicinity of people, etc.);
- Application contexts (email received, websites visited, etc.);
- System contexts (network traffic, status of printers, device battery charge load, etc.).

In (Ríos et. al, 2003; Ríos, 2003), ontologies are used for improving the modeling and handling of contextual information in a context-aware services platform named *WASP* (*Web Architecture for Services Platform*) (Costa et al., 2004a). This approach, illustrated in figure 3.12, is summarized in the sequel.

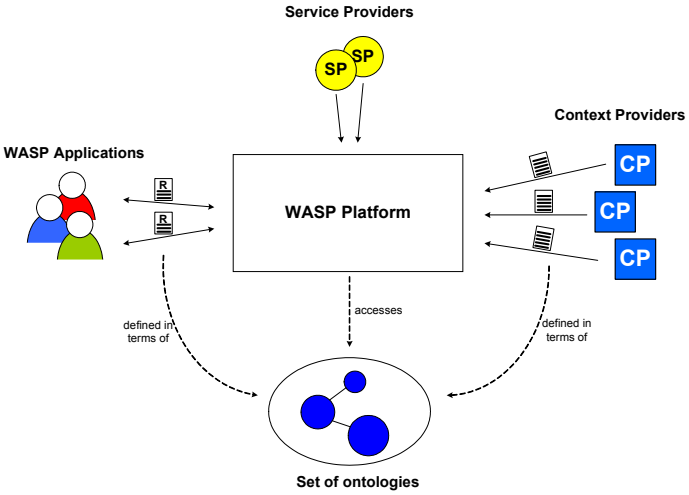


Figure 3-12 The ontology-based version of the WASP platform (from Ríos, 2003)

The objective of the WASP platform is to serve as broker between four different types of entities, namely, *context providers*, *service providers*, *WASP applications* and *semantic web ontologies.*

Context Providers (CP) are responsible for making contextual information available to the other entities interaction with the platform. Examples of Context Providers include physical sensors and software agents. The architecture must be open to new kinds of third-party providers. These providers may supply information using different protocols and/or languages using different syntaxes. An example of a message sent to the platform by a CP would be: *"The visitor John Smith is inside the Chiaroscuro*

gallery of the Rijksmuseum". This message can be considered a description relating the resource *John Smith* (identified via a URI as a record in some database) to another resource (the *Chiaroscuro* gallery) via the property *inside*.

The platform is open to third-party *Service Providers (SP)* interested in offering services to the users of the platform. Depending on the user requirements and context, the platform must support discovery and publishing of services. An example of a simple service is an instant messenger/SMS service provided by a third party SP.

Another objective of the platform is to provide to WASP applications facilities for reacting to their dynamic environment. This is accomplished by allowing the applications to describe to the platform what actions should be taken (e.g., execute a service) in case a situation associated with a context holds. An example of a WASP subscription would be *"**if** a visitor enters the Chiaroscuro gallery in the Rijksmuseum **then** he should be notified via a SMS service about the free Rembrandt Calendar Gift"*, or *"**if** a visitor enters a museum, cinema or theatre **then** a silent mode instruction must be sent to his mobile phone via a mobile phone control service"*.

Ontologies in this case are logical theories that play the role of semantic domains. In this way, context providers and WASP applications can interoperate because they use the same set of interrelated ontologies to define the semantics of both contextual information messages and application subscriptions. For instance, the meaning of *visitor*, *museum*, *museum gallery*, and of the predicate *inside* can be defined in terms of: (a) more general ontologies that, for instance, define the properties of physical objects and places; (b) domain-specific ontologies that define characteristics of museums, galleries and works of art. By reasoning on these ontologies, the platform can detect inconsistencies (e.g., a person cannot be in two disjoint places at the same time), and exhibit a more intelligent behavior, by deriving knowledge from the factual knowledge available (e.g., a visitor is in *a place if he is in any part of that place).*

In (Ríos, 2003), some applications of context-aware services using the ontology-based version of the WASP platform are presented. These include: (i) an *airport information application* offering services related to flight information; (ii) an *event advisors* which notifies users about upcoming events that match their personal interests; (iii) *a friend finder* application that notifies an user when he is close to or inside the same place as one of his friends. Both (ii) and (iii) make use of a *Tourism Ontology*, depicted in figure 3.13[23].

[23] This picture is an exactly copy from (Rios, 2003), in which a case tool is used to generate a UML syntax for the original OWL representation. The notation *p:X* in this specification symbolizes that X is a *property* in the OWL sense of the term.

The sense of the term ontology adopted in the Semantic web vision is similar to the one adopted by the AI and domain engineering communities, i.e., ontology as an engineering artifact consisting of a formal structure of concepts and relations among concepts, and a set of axioms that both constrains the interpretation of this structure and affords the derivation of knowledge from the factual knowledge represented in the structure.

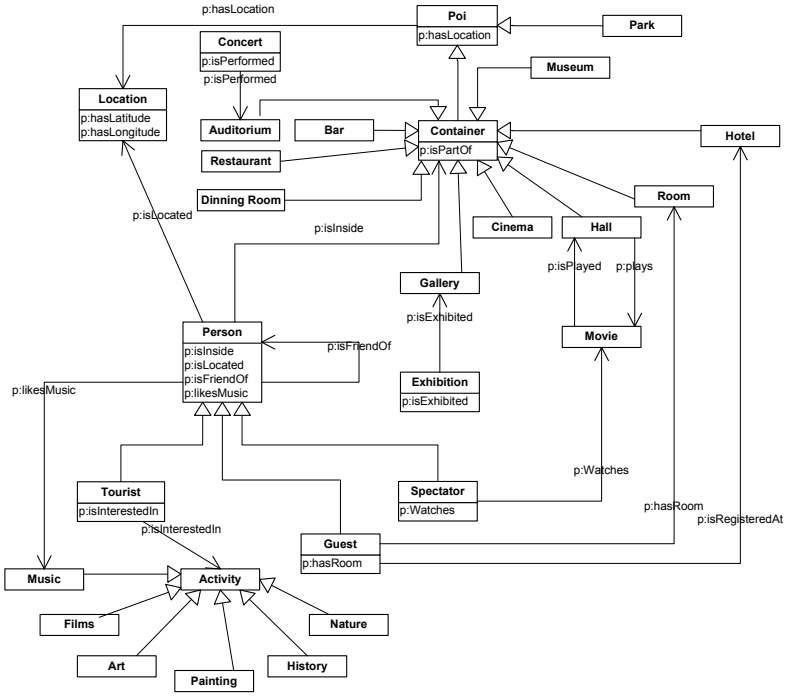


Figure 3-13 A Tourism Ontology referenced by context-aware applications in the ontology-based version of the WASP platform (exactly copy from Ríos, 2003)

3.3 Terminological Clarifications and Formal Characterizations

In the information systems community, the term ontology has a clear meaning, which is akin to the original definition (D2) given in philosophy. In the works of (Wand & Storey & Weber, 1999; Opdahl & Henderson-Sellers, 2001), for example, ontology is clearly used as a system of categories, which is independent of language and state of affairs. In contrast, in the majority of the works in other areas of computer science, the term is used to denote a class of artifacts with a spectrum of divergent characteristics. To illustrate this terminological confusion, figure 3.14 from

(Smith & Welty, 2001), depicts a wide range of different artifacts that have been termed ontologies in the literature. One can observe that these specifications range from simple catalogs containing, for instance, the products that a company sells, to a lexicon of terms with natural language definitions (thesaurus), to formal logical theories.

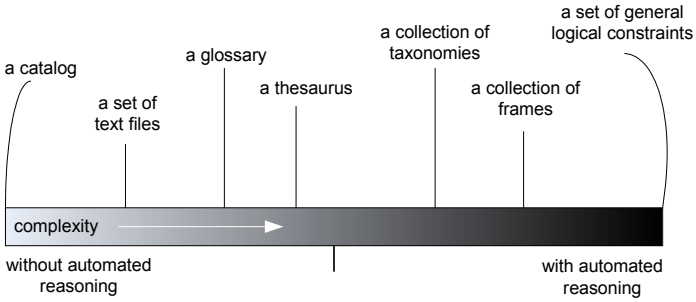


Figure 3-14 Different sorts of specifications classified as ontologies in the computer science literature (from Smith & Welty, 2001)

In (Guarino & Giaretta, 1995), the authors discuss seven different interpretations of the word ontology as used in AI and, after a careful analysis, restrict its possible use to three sound interpretations:

(a). ontology as a representation of a conceptual system that is characterized by specific logical properties (special type of logical theory containing only necessarily true formulas);
(b). ontology as a specification of an ontological commitment;
(c). ontology as a synonym of conceptualization.

Many of the specifications accounted in the classification of figure 3.14 are indeed ontologies in sense (a). However, another share of them complies only with a sense of the term, which was deemed in Guarino & Giaretta's analysis insufficient to qualify as an ontology, namely, ontology as a representation of a conceptual system that is *characterized by specific purposes*. These different senses, as well as their interrelations are discussed in depth in the remaining of this section, in which we establish a precise definition for these terms as they are used in the remaining of this work.

3.3.1 Ontology and Conceptualization

One of the most cited definitions for the term ontology in the Computer Science literature is the one given by Tom Gruber in (Gruber, 1995): *"An Ontology is a explicit representation of a conceptualization"*.

As discussed in (Guarino & Giaretta, 1995; Guarino, 1995), the way this definition was originally proposed by Gruber, it is a problematic one as it relies on a notion of conceptualization that is purely extensional, namely

that of (Genesereth & Nilsson, 1987). Moreover, as argued in (Smith & Welty, 2001), it is too open for interpretations as all those different types artifacts in figure 3.14 comply with this definition. Nonetheless, it captures an intuitive idea, which remains true for the sense of ontology which is employed in the great majority of the works referring to ontology in artificial intelligence, domain engineering and the semantic web.

In chapter 2, we have used figure 3.15 to illustrate the relationships between a conceptualization, a language, a domain abstraction[24] (representing part of some state of affairs in reality articulated according to the conceptualization) and a specification in the language which represents this domain abstraction.

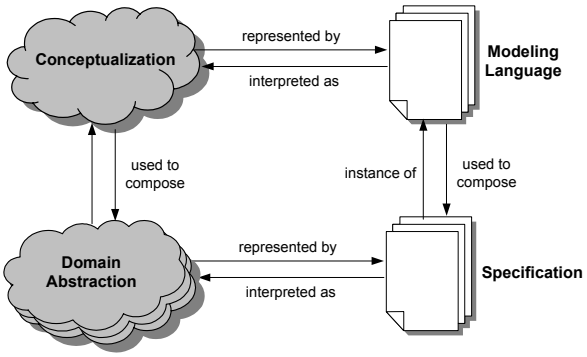


Figure 3-15 Relations between conceptualization, domain abstraction, modeling language and specification

In (Guarino, 1998), the author points out that the sense in which ontology is used in philosophy as a system of categories accounting for a certain vision of the world is akin to what we name a *conceptualization* in figure 3.15. In the philosophical reading, an ontology is independent of language and of particular epistemic state or state of affairs. On one hand, the same ontology could be represented in different languages, For example, the words *orange*, *arancia*, *laranja* and *sinasappel* refer to exactly the same ontological entity, namely the natural kind denoted by these terms. On the other hand, an ontology is neutral w.r.t. the actual existence of a particular orange *a*, but in contrast, it stipulates that if there is a situation in which *a* exists as an orange then *a* also exists as a fruit in this situation. Finally, although we can have an epistemic state of an agent expressing his uncertainty whether *a* is an *orange OR a lemon*, there is no such a thing as an

[24] In chapter 2, we have used the term *model* instead of *domain abstraction* since it is the most common term in conceptual modeling. In this session, exclusively, we adopt the latter in order to avoid confusion with the term (logical) model as used in logics and tarskian semantics. The term logical model here, in turn, bears no relation to the term *logical model* in databases as used in section 3.2.1 of this chapter.

ontological optional property, or something in reality that has the property of being *either an orange or a lemon*.

However, in order to reason about the characteristics of a conceptualization, we must have it captured in some concrete form. In this section, we use a minimum concrete representation of a conceptualization, which is still precise and useful for the purpose of the discussion. The idea is to characterize a conceptualization as an *intensional structure* which, hence, encompasses all state of affairs considered, and which is independent of a particular language vocabulary. This notion of a conceptualization has been proposed in (Guarino, 1998) and can be formally defined as follows:

Definition 3.1 (conceptualization): a conceptualization $\mathcal{C}$ is an intensional structure $\langle \mathcal{W}, \mathcal{D}, \mathfrak{R} \rangle$ such that $\mathcal{W}$ is a (non-empty) set of possible worlds, $\mathcal{D}$ is the domain of individuals and $\mathfrak{R}$ is the set of n-ary relations (concepts) that are considered in $\mathcal{C}$. The elements $\rho \in \mathfrak{R}$ are intensional (or conceptual) relations with signatures such as $\rho^n : \mathcal{W} \rightarrow \wp(\mathcal{D}^n)$, so that each n-ary relation is a function from possible worlds to n-tuples of individuals in the domain.

■

For instance, we can have ρ accounting for the meaning of the natural kind apple. In this case, the meaning of apple is captured by the intensional function ρ, which refers to all instances of apples in every possible world.

Definition 3.2 (intended world structure): For every world $w \in \mathcal{W}$, according to $\mathcal{C}$ we have an *intended world structure* $\mathcal{S}_w\mathcal{C}$ as a structure $\langle \mathcal{D}, \mathcal{R}_w\mathcal{C} \rangle$ such that $\mathcal{R}_w\mathcal{C} = \{\rho(w) \mid \rho \in \mathfrak{R}\}$.

■

More informally, we can say that every intended world structure $\mathcal{S}_w\mathcal{C}$ is the characterization of some state of affairs in world w deemed admissible by conceptualization $\mathcal{C}$. From a complementary perspective, $\mathcal{C}$ defines all the admissible state of affairs in that domain, which will be represented by the set $\mathcal{S}_\mathcal{C} = \{\mathcal{S}_w\mathcal{C} \mid w \in \mathcal{W}\}$.

Let us consider now a language $\mathcal{L}$ with a vocabulary $\mathcal{V}$ that contains terms to represent every concept in $\mathcal{C}$.

Definition 3.3 (logical model): A logical model for $\mathcal{L}$ can be defined as a structure $\langle \mathcal{S}, \mathcal{I} \rangle$: $\mathcal{S}$ is the structure $\langle \mathcal{D}, \mathcal{R} \rangle$, where $\mathcal{D}$ is the domain of

individuals and $\mathcal{R}$ is a set of extensional relations; $I:\mathcal{V}\rightarrow\mathcal{D} \cup \mathcal{R}$ is an interpretation function assigning elements of $\mathcal{D}$ to constant symbols in $\mathcal{V}$, and elements of $\mathcal{R}$ to predicate symbols of $\mathcal{V}$. A model, such as this one, fixes a particular extensional interpretation of language $\mathcal{L}$.

■

Definition 3.4 (intensional interpretation): Analogously, we can define an intensional interpretation by means of the structure $\langle \mathcal{C},\mathfrak{I}\rangle$, where $\mathcal{C} = \langle \mathcal{W},\mathcal{D},\mathfrak{R}\rangle$ is a conceptualization and $\mathfrak{I}:\mathcal{V} \rightarrow \mathcal{D} \cup \mathfrak{R}$ is an intensional interpretation function which assigns elements of $\mathcal{D}$ to constant symbols in $\mathcal{V}$, and elements of $\mathfrak{R}$ to predicate symbols in $\mathcal{V}$.

■

In (Guarino, 1998), this intensional structure is named the *ontological commitment* of language $\mathcal{L}$ to a conceptualization $\mathcal{C}$. We therefore consider this intensional relation as corresponding to the *represents* relation depicted in Ullmann's triangle in figure 3.16 depicted below (see discussion in section 2.1.4).

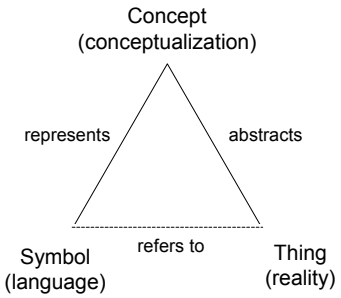


Figure 3-16 Ullmann's Triangle: the relations between a thing in reality, its conceptualization, and a symbolic representation of this conceptualization

Definition 3.5 (ontological commitment): Given a logical language $\mathcal{L}$ with vocabulary $\mathcal{V}$, an *ontological commitment* $\mathcal{K} = \langle \mathcal{C},\mathfrak{I}\rangle$, a model $\langle \mathcal{S},\mathcal{I}\rangle$ of $\mathcal{L}$ is said to be compatible with $\mathcal{K}$ if: (i) $\mathcal{S} \in \mathcal{S}_{\mathcal{C}}$; (ii) for each constant c, $\mathcal{I}(c) = \mathfrak{I}(c)$; (iii) there exists a world w such that for every predicate symbol p, $\mathcal{I}$ maps such a predicate to an admissible extension of $\mathfrak{I}(p)$, i.e. there is a conceptual relation ρ such that $\mathfrak{I}(p) = \rho$ and $\rho(w) = \mathcal{I}(p)$. In accordance with (Guarino, 1998), the set $I_k(\mathcal{L})$ of all models of $\mathcal{L}$ that are compatible with $\mathcal{K}$ is named the set of *intended models* of $\mathcal{L}$ according to $\mathcal{K}$.

■

Definition 3.6 (logical rendering): Given a specification X in a specification language $\mathcal{L}$, we define as the logical rendering of X, the logical theory $\mathcal{T}$ that is the first-order logic description of that specification (Ciocoiu & Nau, 2000).

■

In order to exemplify these ideas let us take the example of a very simple conceptualization C such that $\mathcal{W} = \{w,w'\}$, $\mathcal{D} = \{a,b,c\}$ and $\mathfrak{R} = \{person, father\}$. Moreover, we have that $person(w) = \{a,b,c\}$, $father(w) = \{a\}$, $person(w') = \{a,b,c\}$ and $father(w') = \{a,b\}$. This conceptualization accepts two possible state of affairs, which are represented by the world structures $S_wC = \{\{a,b,c\}, \{\{a,b,c\},\{a\}\}$ and $S_{w'}C = \{\{a,b,c\}, \{\{a,b,c\},\{a,b\}\}$. Now, let us take a language $\mathcal{L}$ whose vocabulary is comprised of the terms `Person` and `Father` with an underlying metamodel specification that poses no restrictions on the use of these primitives. In other words, the metamodel specification of $\mathcal{L}$ has the following logical rendering $\mathcal{T}_1$:

$(\mathcal{T}_1)$

```
∃x Person(x)
∃x Father(x)
```

Clearly, we can produce a logical model of $\mathcal{L}$ (i.e., an interpretation that validates the logical rendering of $\mathcal{L}$) but that is not an intended world structure of C. For instance, the model $\mathcal{D}'=\{a,b,c\}$, $person = \{a,b\}$, $father = \{c\}$, and I(`Person`) = $person$ and I(`Father`) = $father$. This means that we can produce a specification using $\mathcal{L}$ whose model is not an *intended model* according to C.

However, we can update the metamodel specification of language $\mathcal{L}$ by adding the following axiom:

$(\mathcal{T}_2)$

```
∃x Person(x)
∃x Father(x)
∀x Father(x)→ Person(x)
```

Contrary to $\mathcal{L}$, the resulting language $\mathcal{L}'$ with the amended underlying metamodel specification $\mathcal{T}_2$ has the desirable property that all its valid specifications have logical models that are *intended world structures of* C.

We can summarize the discussion so far as follows. A domain conceptualization C can be understood as describing the set of all possible state of affairs, which are considered admissible in a given universe of

discourse U. Let $\mathcal{V}$ be a vocabulary whose terms directly correspond to the intensional relations in $\mathcal{C}$. Now, let $\mathcal{X}$ be a *conceptual specification* (i.e., a concrete representation) of universe of discourse U and let $\mathcal{T}_X$ be a logical rendering of $\mathcal{X}$, such that its axiomatization constrains the possible interpretations of the members of $\mathcal{V}$. We call $\mathcal{X}$ (and $\mathcal{T}_X$) an *ideal ontology* of U according to $\mathcal{C}$ iff the logical models of $\mathcal{T}_X$ describe all and only state of affairs which are admitted by $\mathcal{C}$.

The relationships between language vocabulary, conceptualization, ontological commitment and ontology are depicted in figure 3.17 below. This use of the term ontology is strongly related to the third sense (D3) in which the term is used in philosophy, i.e. as *"a theory concerning the kinds of entities and specifically the kinds of abstract entities that are to be admitted to a language system*" (section 3.1).

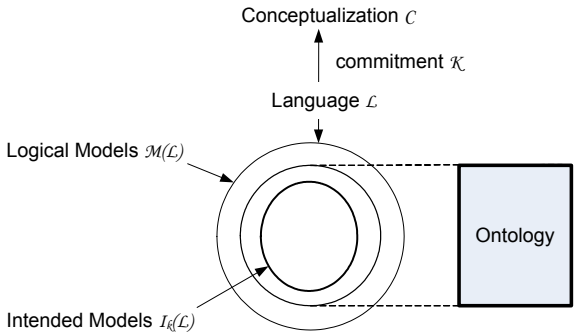


Figure 3-17 Relations between language (vocabulary), conceptualization, ontological commitment and ontology

The logical theory ($\mathcal{T}_2$) described above is, thus, an example of an ontology for the person/father toy conceptualization. The same would apply to the models depicted in figures 2.17, 3.4, 3.5 and 3.13.

As pointed out in (Guarino, 1998), ontologies cannot always be ideal and, hence, a general definition for an (non-ideal) ontology must be given: *An ontology is a conceptual specification that describes knowledge about a domain in a manner that is independent of epistemic states and state of affairs. Moreover, it intends to constrain the possible interpretations of a language's vocabulary so that its logical models approximate as well as possible the set of intended world structures of a conceptualization $\mathcal{C}$ of that domain.*

According to criteria of accuracy, we can therefore give a precise account for the quality of a given ontology. Given an ontology $O_{\mathcal{L}}$ and an ideal ontology $O_{\mathcal{C}}$, the quality of $O_{\mathcal{L}}$ can be measured as the distance between the set of logical models of $O_{\mathcal{L}}$ and $O_{\mathcal{C}}$. In the best case, the two

ontologies share the same set of logical models. In particular, if $O_{\mathcal{L}}$ is the specification of the ontological metamodel of modeling language $\mathcal{L}$, we can state that if $O_{\mathcal{L}}$ and $O_{\mathcal{C}}$ are isomorphic then they also share the same set of possible models. It is important to emphasize the relation between the possible models of $O_{\mathcal{L}}$ and the completeness of language $\mathcal{L}$ (in the technical sense discussed in chapter 2). There are two ways in which incompleteness can impact the quality of $O_{\mathcal{L}}$: firstly, if $O_{\mathcal{L}}$ (and thus $\mathcal{L}$) does not contain concepts to fully characterize a state of affairs, it is possible that the logical models of $O_{\mathcal{L}}$ describe situations that are present in several world structures of $\mathcal{C}$. In this case, $O_{\mathcal{L}}$ is said to *weakly* characterize $\mathcal{C}$ (Guarino, 1998), since it cannot guarantee the reconstruction of the relation between worlds and extensional relations established by $\mathcal{C}$; secondly, if the representation of a concept in $O_{\mathcal{L}}$ is underspecified, it will not contain the axiomatization necessary to exclude unintended logical models. As an example of the latter, we can mention the incompleteness of UML class diagrams w.r.t. classifiers and part-whole relations discussed in chapters 4 and 5 of this work, respectively. In summary, we can state that an ideal ontology $O_{\mathcal{C}}$ for a conceptualization $\mathcal{C}$ of universe of discourse U can be seen as the specification of the ontological metamodel for an ideal language to represent U according to $\mathcal{C}$. For this reason, the adequacy of a language $\mathcal{L}$ to represent phenomena in U can be systematically evaluated by comparing $\mathcal{L}$'s metamodel specification with $O_{\mathcal{C}}$.

By including the third axiom, ($\mathcal{T}_1$) is transformed into an ideal ontology ($\mathcal{T}_2$) of $\mathcal{C}$. One question that comes to the mind is: How can one know that? In other words, how can we systematically design an ontology O that is a better characterization of $\mathcal{C}$. There are two important points that should be called to attention. The first point concerns the language that is used in the representation of these specifications, namely, that of standard predicate calculus. The axiom added to ($\mathcal{T}_1$) to create ($\mathcal{T}_2$) represents a subsumption relation between Person and Father. Subsumption is a basic primitive in the group of the so-called epistemological languages (see 3.4.2), which includes languages such as EER (Elmasri & Weeldreyer & Hevner, 1985) and OWL. It is, in contrast, absent in ontological neutral logical languages such as predicate calculus. By using a language such as OWL to represent a conceptualization of this domain, a specification such as the one in figure 3.18 should be produced. In this model, the third axiom would be automatically included through the semantics of the metamodeling language. Therefore, if a suitable ontology modeling language is chosen, its primitives incorporate an axiomatization, such that the specifications (ontologies) produced using this language will better approximate the intended models of a conceptualization $\mathcal{C}$. Additionally, as

one can notice, subsumption is not a relation which is specific to the represented domain. In contrast, it is a *formal* relation that appears in several different universes of discourse. This feature is compatible with those of primitives that should figure in a general conceptual modeling language.

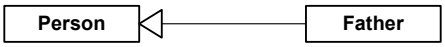


Figure 3-18 Example of a subsumption relation in UML

The second point that should be emphasized is related to the question: *how are the world structures that are admissible to $\mathcal{C}$ determined?* The rationale that we use to decide that are far from arbitrary, but motivated by the laws that govern the domain in reality. In (Bunge, 1977), the philosopher of science Mario Bunge defines the concepts of a *state space* of a thing[25], and a subset of it, which he names a *nomological state space*. The idea is that among all the (theoretically) possible states a thing can assume, only a subset of it is lawful and, thus, is actually possible. Additionally, he defends that the only really possible *facts* involving a thing are those that abide by laws, i.e., those delimited by the nomological state space of thing. As a generalization, if an actual state of affairs consists of facts (Armstrong, 1997), then the set of possible state of affairs is determined by a *domain nomological state space*. In sum, possibility is not by any means defined arbitrarily, but should be constrained by the set of laws that constitute reality. For example, it is law of the domain (in reality) that every Father is a Person. The specification ($\mathcal{T}_2$) is an ideal ontology for $\mathcal{C}$ because it includes the representation of this law of this domain via the subsumption relation between the corresponding representations of father and person. Conversely, if $\mathcal{C}$ included a world structure in which this law would be broken, the conceptualization itself would not be truthful to reality. To refer once more to Ullmann's triangle (figure 3.16), the relation between $\mathcal{C}$ and the *domain nomological state space* is that relation of *abstracts* between a conceptualization and reality.

Now, to raise the level of abstraction, we can also consider the existence of a meta-conceptualization $\mathcal{C}'$, which defines the set of all domain conceptualizations such as $\mathcal{C}$ that are truthful to reality. Our main objective is to define a general conceptual modeling language $\mathcal{L}'$ that can be used to produce domain ontologies such as O_C, i.e., a language whose primitives include theories that help in the formal characterization of a domain-specific language $\mathcal{L}$, restricting its logical models to those deemed

[25] The word Thing is used by Bunge in a technical sense, which is synonymous to the notion of *substantial individual* defined in Chapter 6.

admissible by $\mathcal{C}$. In order to do this, we have to include in $\mathcal{L}'$ primitives that represent the laws that are used to define the nomological world space of meta-conceptualization $\mathcal{C}'$. In this case, these are the general laws that describe reality, and describing these laws is the very business of *formal ontology* in philosophy.

In summary, we defend that the ontology underlying a general conceptual modeling language $\mathcal{L}'$ should be a meta-ontology that describes a set of real-world categories that can be used to talk about reality. Likewise, the axiomatization of this foundational ontology must represent the laws that define that nomological world space of reality. This meta-ontology, when constructed using the theories developed by *formal ontology* in philosophy, is named a *foundational ontology*.

3.3.2 The Ontological Level

When a general conceptual modeling language (or ontology representation language) is constrained in such a way that its intended models are made explicit, it can be classified as belonging to the *ontological level*. This notion has been proposed by Nicola Guarino in (Guarino, 1994), in which he revisits Brachman's classification of knowledge representation formalisms (Brachman, 1979).

In Brachman's original proposal, the modeling primitives offered by knowledge representation formalisms are classified in four different levels, namely: *implementation*, *logical*, *conceptual* and *linguistic* levels.

In the logical level, we are concerned with the predicates necessary to represent the concepts of a domain and with evaluating the truth of these predicates for certain individuals. The basic primitives are propositions, predicates, functions and logical operators, which are extremely general and ontologically neutral. For instance, suppose we want to state that a red apple exists. In predicate calculus we would write down a logical formula such as (F_1) $\exists x(\texttt{apple(x)} \wedge \texttt{red(x)})$.

Although this formula has a precise semantics, the real-world interpretation of a predicate occurring in it is completely arbitrary, since one could use it to represent a property of a thing, the kind the thing belongs to, a role played by the thing, among other possibilities. In this example, the predicates apple and red are put in the same logical footing, regardless of the nature of the concept they represent and the importance of this concept for the qualification of predicated individual. Logical level languages are neutral w.r.t. ontological commitments and it is exactly this neutrality that makes logic interesting to be used in the development of scientific theories. However, it should be used with care and not directly in the development of ontologies, since one can write perfectly correct logical

formulas, but which are devoid of ontological interpretation. For example, the entailment relation has no ontic correlation. Moreover, while one can negate a predicate or construct a formula by a disjuction of two predicates, in reality, there are neither negative nor alternative entities (Bunge, 1977).

In order to improve the "flatness" of logical languages, Brachman proposes the introduction of an *epistemological level* on top of it, i.e., between the logical and conceptual levels in the original classification.

Epistemology is the branch of philosophy that studies "*the nature and sources of knowledge*". The interpretation taken by Brachman and many other of the logicist tradition in AI is that knowledge consists of propositions, whose formal structure is the source of new knowledge. Examples of representation languages belonging to this level include Brachman's own KL-ONE (Brachman & Schmolze, 1985) and its derivatives (including the semantic web languages OIL, DAML, DAML+OIL, RDFS, OWL) as well as object-based and frame-based modeling languages such as EER, LINGO and UML.

The rationale behind the design of epistemological languages is the following: (i) the languages should be designed to capture interrelations between pieces of knowledge that cannot be smoothly captured in logical languages; (ii) they should offer *structuring* mechanisms that facilitate understanding and maintenance, they should also allow for economy in representation, and have a greater computational efficiency than their logical counterparts; (iii) finally, modeling primitives in these languages should represent structural connections in our knowledge needed to justify conceptual inferences in a way that is independent of the meaning of the concepts themselves.

Indeed languages such as UML and OWL offer powerful structuring mechanisms such as classes, relationships (attributes) and subclassing relations. However, if we want to impose a certain *structure* in the representation of formula (F_1), in a language such as UML, we would have to face the following structuring choices: (a) consider that there are instances of apples that can posses the property of being red or, (b) consider that there are instances of red things that can have the property of being apples. Formally we can state either that (F_2) $\exists$`x:Apple.red(x)` as well as (F_3) $\exists$`x:Red.apple(x)`, and both these many-sorted logic formalizations are equivalent to the previous one-sorted axiom. However, each one contains an implicit structuring choice for the sort of the things we are talking about.

The design of epistemological languages puts a strong emphasis on the inferential process, and the study of knowledge is limited to its form, i.e., it is "*independent of the meaning of the concepts themselves*". Therefore, the focus of these languages is more on formal reasoning than on (formal)

representation. Returning to our example, although the representation choice (b) seems to be intuitively odd, there is nothing in the semantics of a UML class or an OWL concept that prohibits any unary predicate such as red or tall to be modeled as such. In other words, since in epistemological languages the semantics of the primitive "sort" is the same as its corresponding unary predicate, the choice of which predicates correspond to sorts is completely left to the user.

In (Guarino, 1994), Guarino points out that structuring decisions, such as this one, should not result from heuristic considerations but instead should be motivated and explained in the basis of suitable *ontological distinctions*. For instance, in this case, the choice of Apple as the sort (a) can be justified by the meta-properties that we are ascribed to the term by the *intended meaning* that we give to it. The ontological difference between the two predicates is that Apple corresponds to a *Natural Kind* whereas Red corresponds to an *Attribution*. Whilst the former applies necessarily to its instances (an apple cannot cease to be an apple without ceasing to exist), the latter only applies contingently. Moreover, whilst the former supplies a *principle of identity* for its instances, i.e., a principle through which we judge if two apples are numerically the same, the latter cannot supply one. However, it is not the case that an object could subexist without obeying a principle of identity (van Leewen, 1991; Gupta, 1980), an idea which is defended both in philosophical ontology (e.g., Quine's dicto "*no entity without identity*" (Quine, 1969)), and in conceptual modeling (e.g., Chen's design rational for ER (Chen, 1976)). Consequently, the structuring choice expressed in (F_3) cannot be justified.

For an extensive discussion on *kinds*, *attributions* and *principles of identity* as well as their importance for the practice of conceptual modeling we refer to chapter 4 of this thesis.

In addition to supporting the justified choice for structuring decisions, the ontological level has important practical implications from a computational point of view. For Instance, one can exploit the knowledge of which predicates hold necessarily (and which are susceptible to change) in the design and implementation of more efficient update mechanisms.

Finally, there are senses in which the term Red can be said to hold necessarily (e.g., "*scarlet is a type of red*" referring to a particular shade of color), and senses in which it carries a principle of identity to its instances (e.g., "*John is a red*" - meaning that "*John is a communist*"). The choice of representing Red as an attribution makes explicit the intended meaning of this predicate, ruling out these two possible interpretations. In epistemological and logical languages, conversely, the intended meaning of a predicate relies on our understanding of the natural language label used.

The position defended in this work is that, in general conceptual modeling languages, the meaning of structuring primitives should be characterized in terms of meta-level conditions that make explicit the ontological commitment made by the modeler when choosing a particular structuring primitive to represent a domain element. For example, in the profile used to create the ontology of figure 2.17, the modeling primitives are tagged with the ontological categories they represent. When choosing to represent Person as a Kind, the user explicitly commits to the axiomatization implied by this representation choice. There are alternative representations in which Person can be considered to hold only contingently, for example, if one considers Person as a synomym for LivingPerson and, let us say, Corpse as synonym for deceasedPerson (figure 2.17). However, when contrasting the two representations we notice that the two usages of the term, although syntactically identical, are semantically and ontologically diverse, since they possess incompatible meta-properties.

In this sense, the approach taken in this thesis departs from the one followed by top-level ontologies such as CyC (Lenat & Guha, 1990) and the IEEE Standard Upper-Level Ontology (SUO)[26]. In these cases, the idea is to create a specification that defines a complete inventory of reality in a way that all domain ontologies could commit to. CyC, for instance, has been under development since 1990 and, only its open source version (OpenCyC[27]), contains currently 6000 concepts and 60.000 assertions about these concepts. Conversely, the position taken here is that the choice of how to represent a certain domain of reality is the responsibility of a community of users. However, the intended meaning embedded in these representation choices should be made explicit via an ontological commitment to a system of meta-level categories, or a *foundational ontology*. In this way, agreement is reinforced on the meta-level and not on the level of individual domain theories. This is far from promoting a scenario where a unique meta-ontology exists. On the contrary, the position defended here is equivalent to the one of (Masolo et. al, 2003a), namely, that there should be a library of *foundational ontologies*, where each of these ontologies (explicitly) commits to different philosophical choices (see, for instance, section 3.4.3), and, consequently, is suitable for different classes of applications. Nonetheless, the stability and universality of these foundational ontologies should be supported by the theoretical developments in formal ontology, and afforded by empirical evidence, either in natural sciences or in the sciences of cognition.

[26] http://suo.ieee.org/
[27] http://www.opencyc.org/

3.3.3 On a suitable Meta-Conceptualization

Ontologies vary in the way they manage to represent an associated conceptualization. An ontology such as the one in figure 2.17 is more accurate than if it were represented in ER, OWL, pUML (Evans & Kent, 1999) or LINGO. This is because the modeling profile which is used commits to a much richer meta-ontology than the ones underlying these other languages. As a consequence, to formally characterize its ontological distinctions, a formal language with higher expressiveness is needed. When the stereotyped modeling primitives of the profile in figure 2.17 are used, an axiomatization in the language of *intensional modal logics* is incorporated in the resulting specification, constraining the intepretation of its terms.

Intensional modal logics are considerably more expressive than *SHIQ* or standard set theory. However, *SHIQ* has interesting properties such as computational tractability and decidability, which are properties that are in general absent in more expressive languages. Likewise, LINGO was designed to facilitate the translation to Object-Oriented implementations. Therefore, in the design of a general conceptual modeling language, the following tradeoff must be recognized. On one side we need a language that commits to a rich foundational ontology. This meta-ontology, however, will require the use of highly expressive formal languages for its characterization, which in general, are not interesting from a computational point of view. On the other side, languages that are efficient computationally, in general, cannot commit to a suitable meta-conceptualization. The obvious question is then: how can we design a general conceptual modeling language according to these conflicting requirements?

The position advocated here is analogous to the one defended in (Masolo et. al, 2003a), namely, that we actually need two classes of languages. On one hand, highly-expressive languages should be used to create strongly axiomatized ontologies that approximate as well as possible to the ideal ontology of the domain. The focus on these languages is on representation adequacy, since the resulting specifications are intended to be used by humans in tasks such as communication, domain analysis and problem-solving. The resulting domain ontologies, named *reference ontologies* in (Guarino, 1998), should be used in an *off-line* manner to assist humans in tasks such as meaning negotiation and consensus establishment. On the other hand, once users have already agreed on a common conceptualization, versions of a reference ontology can be created. These versions are named here *lightweight ontologies.* Contrary to reference ontologies, lightweight ontologies are not focused on representation adequacy but are designed with the focus on guaranteeing desirable

computational properties. Examples of languages suitable for lightweight ontologies include OWL and LINGO. An example of a conceptual modeling language that is suitable for reference ontologies is the one developed throughout this work.

The importance of reference ontologies has been acknowledged in many cases in practice. For instance, (Ríos, 2003) illustrates examples of semantic interoperability problems that can pass undetected when interoperating lightweight ontologies. Likewise, (Fielding et. al, 2004) discusses how a principled foundational ontology can be used to spot inconsistencies and provide solutions for problems in lightweight biomedical ontologies. As a final example, the need for methodological support in establishing precise meaning agreements is recognized in the *Harvard Business Review* report of October 2001, which claims that *"one of the main reasons that so many online market makers have foundered [is that] the transactions they had viewed as simple and routine actually involved many subtle distinctions in terminology and meaning"*.

Once we have made clear that the meta-ontology developed throughout this work should be a foundational one, an important issue still remains to be addressed, namely, which kind of foundational ontology one should commit to. In (Strawson, 1959), the philosopher Peter Strawson draws a distinction between two different kinds of ontological investigation, namely, *descriptive* and *revisionary methaphysics*. Descriptive metaphysics aims to lay bare the most general features of the conceptual scheme that are in fact employed in human activities, which is roughly that of common sense. The goal is to capture the ontological distinctions underlying natural language and human cognition. As a consequence, the categories refer to cognitive artifacts more or less depending on human perception, cultural imprints and social conventions (Masolo et. al, 2003a), and do not have necessarily to agree on the principles advocated by the natural sciences. Nonetheless, the very existence of these categories can be empirically uncovered by research in cognitive sciences (Keil, 1979; Xu & Carey, 1996; Mcnamara, 1986) in a manner that is analogous to the way philosophers of science have attempted to elicit the ontological commitments of the natural sciences

Revisionary metaphysics, conversely, is prepared to make departures from common sense in light of developments in science, and considers linguistic and cognitive issues of secondary importance (if considered at all). The following example, from (Masolo et. al, ibid.), exemplifies these different approaches. Common sense distinguishes between *things* (*spatial objects* like a car, a city and the moon) and *events* (*temporal objects* like business processes, birthday parties and football games). According to the relativity theory, however, time is viewed as another dimension of objects

on a par with the spatial dimensions. As a consequence, revisionist researchers propose that the common sense distinction between things and events should be regarded as an (ontologically irrelevant) historical and cognitive accident, and that it should be abandoned in favor of an ontology of processes (Sowa, 2000; Whitehead, 1978).

In summary, whilst a descriptive ontology aims at giving a correct account of the categories underlying human common sense, a revisionary ontology is committed to capture the intrinsic nature of the world in a way that is independent of conceptualizing agents. Nonetheless, the taxonomies of objects produced by both approaches can be shown to be in large degree compatible with each other, if only we are careful to take into account the different granularities at which each operates (Smith & Brogaard, 2002).

In order to motivate a choice for a foundational ontology that a general conceptual modeling language should commit to, we first must investigate the definition and purposes of conceptual modeling and conceptual specifications. In a seminal paper, John Mylopoulos (Mylopoulos, 1992) provides the following definition, which highlights some aspects of the discipline and the produced representations that are of great relevance for the discussion carried out here. First, he defines conceptual modeling as *"the activity of formally describing some aspects of the* ***physical*** *and* ***social*** *world around us for purposes of* ***understanding*** *and* ***communication****."* This passage highlights that conceptual modeling is about the modeling of reality and not about modeling a computational system. In another part of the text he states that *"conceptual modelling supports structuring and inferencial facilities that are* ***psychologically grounded****. After all, the descriptions that arise from conceptual modelling activities are intended to be used* ***by humans, not machines"****.* This is an important point that also emphasizes the precedence of truthfulness to reality over features such as computational efficiency and tractability of the produced representations. Moreover, it draws attention to the importance of considering human-oriented issues, such as the need to produce representations that are pragmatically efficient and are psychologically grounded and amenable to human cognition. Finally, he summarizes these aspects in the following adequacy requirement for conceptual modeling languages:*"The adequacy of a conceptual modelling notation rests on its contribution to the construction of models of reality that promote a common understanding of that reality among their human users."*

The main objective of conceptual specifications (in particular domain ontologies) in computer science is to support tasks such as communication, domain learning and problem solving in disciplines such as domain engineering and database schema integration. Moreover, in cases such as inter-organizational service interoperability, information brokering and intelligent search in the Semantic Web, the ontologies produced have an

intrinsic social nature. In the majority of cases, these ontologies represent conceptualizations that are afforded by our common sense view of reality. For this reason, in this work we defend the idea that a general conceptual modeling language should commit to a *descriptive foundational ontology*. Consequently, the foundational ontology presented here should be understood as a descriptive theory of *a priori* distinctions, focused on entities of the so-called *mesoscopic* level, i.e., the level of human experience. That is to say that the categories in our foundational ontology focus on meta-properties of everyday objects such as apples, people, cars, chairs and insurance claims but neither on microscopic or macroscopic entities. For instance, it is outside the scope of this thesis to consider ontological problems such as the ones discussed in (Lowe, 2001), which arise when characteristics of entities of the atomic and subatomic levels are considered.

3.4 Final Considerations

In the course of this chapter we have discussed the reasons that have historically motivated the use of philosophical ontology in computer science disciplines such as information systems, domain engineering and knowledge representation. A common aspect of these disciplines is the need to:

(i) promote reuse in a higher level of abstraction aiming at the maximization of the reuse of domain models;
(ii) produce domain specifications that are truthful to reality.

The main contribution of philosophical ontology in the accomplishment of these goals is the set of conceptual tools that have been developed along the years for constructing these general systems of categories. In philosophy, an ontology is committed only to the truth in the classical sense, i.e., it is meant to represent knowledge of reality in a way that is independent of a particular use one makes of it. Since the very task of semantic interoperation (e.g., database schema and framework integration, service interoperability) is to find a common conceptualization that different representations agree on, the potential benefit from philosophical ontology becomes evident or, how could this common conceptualization be achieved if not through a task-independent model of the underlying reality that different representations refer to?

The relations between the various senses in which the term ontology has been used in philosophy and computer science can be summarized as follows:

1. *Formal Ontology*, as conceived by Husserl, is part of the discipline of Ontology in philosophy (sense D1), which is, in turn, the most important branch of metaphysics;

2. Formal Ontology aims at developing general theories that accounts for aspects of reality that are not specific to any field of science, be it physics or conceptual modeling (sense D2);

3. These theories describe knowledge about reality in a way, which is independent of language, of particular states of affairs (states of the world), and of epistemic states of knowledgeable agents. In this thesis, these language independent theories are named (meta) *conceptualizations*. The representation of these theories in a concrete artifact is a *foundational ontology*;

4. A foundational ontology tries to characterize as accurately as possible the conceptualization it commits to. Moreover, it focuses on representation adequacy regardless of the consequent computational costs, which is not actually a problem since the resulting model is targeted at human users.

5. A foundational ontology can be used to provide real-word semantics for general *conceptual modeling languages*, and to constrain the possible interpretations of their modeling primitives. An ontology can be seem as the metamodel specification for an ideal language to represent phenomena in a given domain in reality, i.e., a language which only admits specifications representing possible state of affairs in reality (related to sense D3).

6. Finally, suitable general conceptual modeling languages can be used in the development of reference *domain ontologies*, which, in turn, among many other purposes, can be used to characterize the vocabulary of *domain-specific languages*.